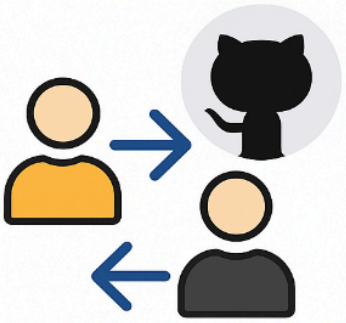
Universidad Tecnológica Nacional

Guía de Uso de Git y GitHub



Colaboración, Control de Versiones e Integración en Entornos Educativos

Material de apoyo para la Cátedra Organización Empresarial – TUP UTN (Modalidad a Distancia)

Cátedra: Organización Empresarial
Titular: Prof. Gabriela Martínez
Autor: Prof. Mario Raúl López
Año: 2025

📘 Resumen

La presente guía tiene como propósito introducir de manera práctica y didáctica el uso de **Git** y **GitHub** como herramientas esenciales para el trabajo colaborativo, la trazabilidad de los proyectos y la organización empresarial del desarrollo de software.
A través de ejemplos guiados —como el proyecto *Hola Mundo!* y el caso real de tres estudiantes trabajando con GitHub desde Google Colab— se busca vincular la teoría de la gestión empresarial con las prácticas modernas de desarrollo distribuido.

Este material integra conceptos de control de versiones, gestión de ramas, resolución de conflictos, documentación compartida y revisión por pares, aplicados al contexto educativo de la **Tecnicatura Universitaria en Programación a Distancia (UTN)**.

🎯 Objetivos de la Guía

1. **Comprender** la importancia del control de versiones en la organización de proyectos de software.
2. **Introducir** los conceptos fundamentales de Git y GitHub, diferenciando su rol local y remoto.
3. **Aplicar** los comandos básicos de Git para registrar, sincronizar y compartir código.
4. **Experimentar** con flujos de trabajo colaborativos mediante el uso de ramas y *pull requests*.
5. **Promover** el uso de entornos virtuales como Google Colab para la integración práctica del control de versiones.
6. **Relacionar** los principios de gestión empresarial con la colaboración y documentación en proyectos de programación.

🧠 Estructura de la Guía

Sección	Contenido principal
1. Introducción al control de versiones	Conceptos, ventajas y escenarios reales
2. Fundamentos de Git	Ciclo de vida de un archivo, comandos esenciales
3. GitHub y la nube colaborativa	Repositorios, ramas, commits y sincronización
4. Ejemplo práctico: Hola Mundo en Colab	Guía paso a paso para principiantes
5. Caso real: Hugo, Paco y Luis	Trabajo colaborativo y trazabilidad en equipo

6. Buenas prácticas	Documentación, seguridad y comunicación en proyectos
7. Conclusiones y bibliografía	Cierre conceptual y fuentes recomendadas

1. Introducción: El valor del control de versiones en la organización del trabajo

En el ámbito del desarrollo de software, la capacidad de registrar, revisar y coordinar los cambios realizados por distintos integrantes de un equipo es fundamental para garantizar la calidad del producto final.
Esta necesidad no es exclusiva del mundo empresarial: también se aplica al entorno académico, donde estudiantes, docentes y tutores trabajan de manera distribuida, intercambiando materiales, scripts, informes o prácticas de programación.

En este contexto, **Git** y **GitHub** se consolidan como herramientas esenciales para la **organización, trazabilidad y colaboración** dentro de proyectos individuales o grupales.

- **Git**, creado por Linus Torvalds en 2005, es un sistema de **control de versiones distribuido (DVCS)** que permite registrar cada modificación del código, conservar su historial y volver a estados anteriores cuando sea necesario.
- **GitHub**, por su parte, es una **plataforma en la nube** que amplía las capacidades de Git, integrando funcionalidades de trabajo en equipo, revisión de código, documentación compartida y gestión de proyectos.

Desde una perspectiva organizacional, estas herramientas aportan **eficiencia, transparencia y control**, tres pilares esenciales en cualquier estructura moderna, ya sea empresarial o educativa.
Implementar Git y GitHub en el proceso formativo permite a los estudiantes **adoptar metodologías de trabajo reales**, similares a las que se aplican en empresas tecnológicas, promoviendo competencias como la comunicación efectiva, la planificación y la gestión de recursos digitales.

El propósito de esta guía es **introducir el uso de Git y GitHub de forma gradual y práctica**, acompañando al estudiante en su primer contacto con estas herramientas.
Se brindan explicaciones paso a paso, ejemplos desarrollados en **Google Colab**, y un caso real donde tres estudiantes colaboran para resolver un **Trabajo Práctico Integrador (TPI)**, utilizando repositorios compartidos y flujos de trabajo profesionales.

De este modo, el control de versiones deja de ser solo un aspecto técnico del desarrollo para convertirse en una **herramienta de organización y liderazgo**, alineada con los objetivos de la materia **Organización Empresarial** dentro de la **Tecnicatura Universitaria en Programación a Distancia**.

2 Fundamentos de Git

2.1 ¿Qué es Git, en pocas palabras?

Git es un **sistema de control de versiones distribuido (DVCS)**. Cada desarrollador trabaja con su **repositorio local** que contiene todo el historial del proyecto; esto aporta resiliencia (no dependés de un servidor central) y flexibilidad para trabajar offline.

2.2 Ciclo de vida de un archivo en Git (los 3 estados)

Git modela los cambios de los archivos mediante **tres estados** principales. Entender estos estados es fundamental para usar correctamente los comandos.

1. **Working Directory (Directorio de trabajo)**
    - Es la copia de los archivos tal como los ves y editas en tu sistema de archivos (tu carpeta del proyecto).
    - Cuando abres o modificas un archivo con el editor, esos cambios quedan en este estado.
    - Ejemplo: editás `hola.py` en tu editor.

2. **Staging Area (Área de preparación o index)**
    - Es una “zona intermedia” donde marcás exactamente qué cambios querés incluir en el próximo commit.
    - Se configuran con `git add`. Esto permite agrupar varios cambios en commits lógicos.
    - Ejemplo: `git add hola.py` añade la versión actual de `hola.py` al staging.

3. **Repository (Repositorio local / Git directory)**
    - Aquí se guarda el historial (los commits). Un **commit** es una fotografía del contenido indexado (staging) con metadatos (autor, fecha, mensaje).
    - Se crea con `git commit`.
    - Los commits pueden luego subirse a un repositorio remoto (GitHub) con `git push`.

Flujo típico (resumido):
editar archivo → `git add` (staging) → `git commit` (repositorio local) → `git push` (repo remoto).

2.3 Comandos esenciales: qué hacen y ejemplos prácticos

A continuación los comandos más importantes, ordenados por el flujo de trabajo.

Inicialización y configuración

`git init`

Inicializa un repositorio Git en la carpeta actual (crea `.git`).

Ejemplo:

```
git init
git config --global user.name "Tu Nombre"
git config --global user.email "tu.email@ejemplo.com"
```

Define la identidad que aparecerá en los commits.

Estado y seguimiento

`git status`

Muestra el estado actual: archivos modificados, staged, sin seguimiento.

```
git status
git add <archivo>
```

Añade un archivo (o cambios) al **staging area**.

```
git add hola.py
git add .            # agrega todos los cambios (usar con cuidado)
git diff
```

Muestra diferencias no staged entre working directory y staging.

```
git diff            # cambios no añadidos
git diff --staged   # diferencias entre staging y último commit
```

Commit (guardar en el repo local)

`git commit -m "Mensaje claro y descriptivo"`

Crea un commit con los cambios del staging area.

```
git commit -m "Agregar hola.py con mensaje inicial"
git commit -am "Mensaje"
```

Atajo: añade y commitea archivos rastreados (no añade archivos nuevos no rastreados).

```
git commit -am "Actualizar README y arreglar typo"
```

Historial

`git log`

Muestra el historial de commits.

```
git log --oneline --graph --decorate
```

Ramas (branches)

`git branch`

Lista ramas locales.

```
git branch           # lista
git branch nueva-rama
```

`git checkout -b nueva-rama`
Crea y cambia a una nueva rama.
```
git checkout -b feature/analisis
```
`git merge rama`
Fusiona `rama` en la rama actual (resolver conflictos si aparecen).
```
git checkout main
git merge feature/analisis
```

Conexión con repositorio remoto (GitHub)

`git remote add origin <URL>`
Conecta el repo local con el remoto (solo la primera vez).
`git remote add origin https://github.com/usuario/Repo.git`
`git push`
Envía commits locales al remoto.
```
git push origin main
```
`git pull`
Trae cambios desde el remoto y los mezcla con tu rama local.
```
git pull origin main
```
`git clone <URL>`
Clona un repositorio remoto a una carpeta local (copia completa del historial).
```
git clone https://github.com/usuario/Repo.git
```

Revisión y corrección

`git revert <commit>`
Crea un nuevo commit que deshace los cambios de `<commit>` (seguro para histórico público).
`git reset --hard <commit>`
Mueve HEAD y sobrescribe working directory (peligroso: puede perder trabajo local no guardado).

2.4 Ejemplo completo (mini flujo de trabajo)

```
# clonar repo del equipo
git clone https://github.com/tuUsuario/TPI_Estadistica.git
cd TPI_Estadistica

# crear rama para análisis de datos
git checkout -b feature/analisis-sturges

# crear/editar script con R o Python
%%writefile scripts/analisis.R
```

```
# (código R que calcula tablas y gráficos)
git add scripts/analisis.R
git commit -m "Agregar análisis de horas de estudio - tablas Sturges y gráficos"
git push origin feature/analisis-sturges

# en GitHub: abrir Pull Request desde feature/analisis-sturges hacia main
# revisión, aprobación y merge por Luis (por ejemplo)
```

3 GitHub y la nube colaborativa

GitHub funciona como plataforma de hospedaje de repositorios Git en la nube, permitiendo que equipos distribuidos colaboren en proyectos de forma estructurada.
Sus funcionalidades más relevantes incluyen:

- **Repositorios remotos**: se almacenan en servidores de GitHub, facilitando el respaldo y la visibilidad del código.
- **Commits y ramas (branches)**: cada desarrollador puede crear una rama para trabajar sin afectar la versión principal (*main* o *master*), y luego fusionar los cambios cuando estén listos.
- **Pull Requests (PRs)**: mecanismo de revisión de código donde otros miembros del equipo pueden comentar, aprobar y fusionar cambios, garantizando calidad y coordinación.
- **Issues y acciones (Actions)**: los *issues* permiten rastrear tareas, bugs o mejoras; las *Actions* automatizan flujos de trabajo (por ejemplo, tests, builds, despliegues).
- **Visibilidad y trazabilidad**: GitHub registra quién hizo qué, cuándo y por qué. Esto es clave para ambientes académicos y empresariales, donde la responsabilidad y la documentación importan.

La sinergia entre Git (el sistema local) y GitHub (la plataforma remota) permite a los equipos aplicar metodologías ágiles, trabajar de forma asincrónica e incrementar la calidad del software mediante revisión colaborativa.

4. Ejemplo práctico: Hola Mundo en Colab

Resumen corto del flujo

1. Crear un **repositorio vacío** en GitHub.
2. Generar un **Personal Access Token (PAT)** para poder `push` desde Colab.
3. Abrir Google Colab, configurar tu identidad Git, **clonar** el repo.

4. Crear `hola.py`, `git add`, `git commit` y `git push` (subirlo a GitHub).
5. Verificar en GitHub que el archivo esté online.

Detalle paso a paso

4.1) Crear el repositorio en GitHub (rápido)

1. Ir a https://github.com y loguearte.
2. Hacer clic en **New repository**.
3. Poner nombre: `HolaMundo` (u otro que quieras).
4. Seleccionar **Public** (o Private si preferís) y marcar **Add a README** (opcional).
5. Crear el repo con **Create repository**.

Copiá la URL HTTPS del repo; será algo como:
`https://github.com/tu-usuario/HolaMundo.git`

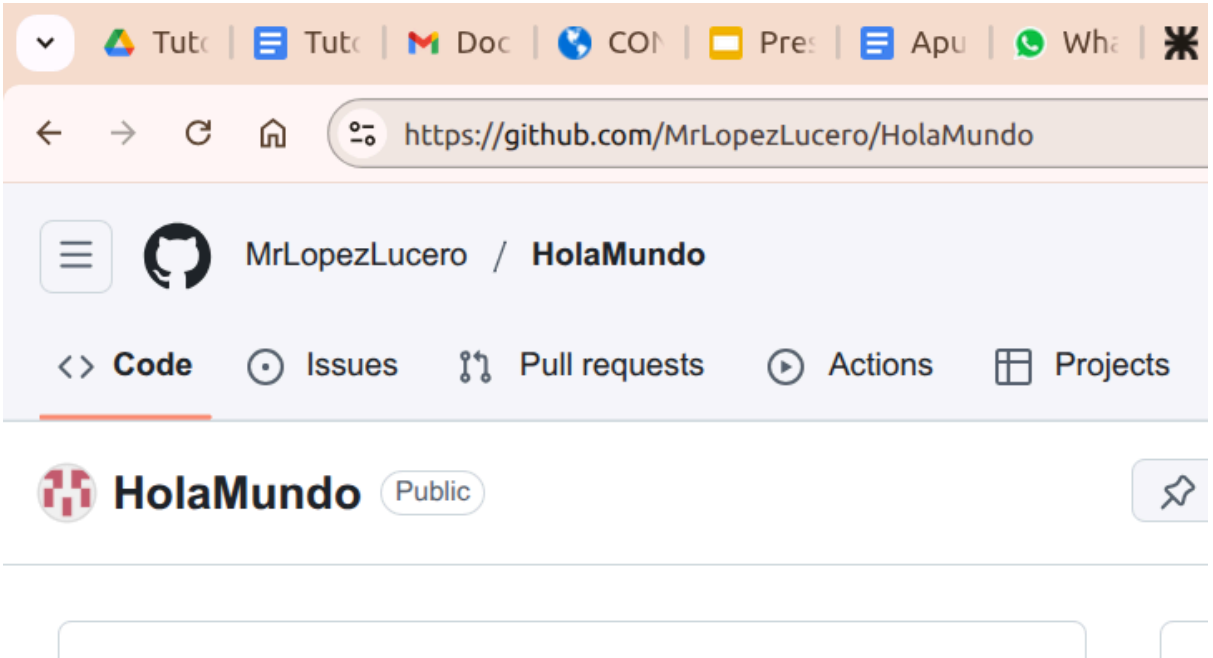


Fig. 1: Creación del nuevo Repositorio “HolaMundo”
Fuente: Elaboración propia.

4.2) Generar un Personal Access Token (PAT) en GitHub

(esto sirve para autenticarnos desde Colab; **no uses tu contraseña**)

1. En GitHub, ir a **Settings** → **Developer settings** → **Personal access tokens**.
2. Elegir **Generate new token (classic)** o la opción disponible.
3. Darle un nombre (ej: `Colab Token`) y expiración si querés.
4. Marcar el permiso **repo** (para acceso a repositorios) — es suficiente.
5. Generar y **copiar el token** (¡se muestra una sola vez!).

IMPORTANTE: Trata el token como una contraseña. No lo pegues en sitios públicos. Podés revocarlo después si querés.

🔐 Cómo generar un Personal Access Token (PAT) en GitHub (versión nueva 2025)

✅ Opción actual: "Fine-grained personal access tokens"

GitHub reemplazó los "classic tokens" por una versión más segura y moderna.

🪜 Pasos actualizados:

1. Iniciá sesión en tu cuenta de **GitHub**.
2. En la esquina superior derecha, hacé clic en tu **foto de perfil** → **Settings**.
3. En el menú lateral izquierdo, **bajá hasta el final** y hacé clic en:
   👉 **"Developer settings"**.
   (A veces tarda en cargar o está oculto al final del panel, debajo de "Integrations")

   Si **no lo ves**, abrí directamente este enlace:
   🔗 https://github.com/settings/tokens
4. Dentro de "Developer settings", elegí la opción:
   → **Personal access tokens** → **Fine-grained tokens**

5. Presioná el botón azul **“Generate new token”** (puede decir *Generate new token (fine-grained)*).



Fig. 2: Resumen generación del Personal Access Token
Fuente: Elaboración propia asistido por IA (ChatGPT v.5).

⚙️ Configuración del token

1. **Token name:**
   Escribí algo como: `Colab Git Access`.

2. **Expiration:**
   Elegí “No expiration” (para evitar que caduque rápido) o un plazo corto si querés seguridad extra.

3. **Resource owner:**
   Dejá tu usuario (si usás tu cuenta personal, no organización).

4. **Repository access:**
   Elegí: **“Only select repositories”**
   Luego marcá tu repositorio, por ejemplo `HolaMundo`.

5. **Repository permissions:**
   En la lista desplegable, en la categoría **Repository permissions**, activá:
   - **Contents → Read and write**

6. Finalmente, hacé clic en **Generate token**.

GitHub te mostrará el token una sola vez, algo así como:
`github_pat_11A1B1CDEFGHijklmnop1234567890ABC`

7. 📋 **Cópialo y guárdalo** enseguida, porque después no se puede volver a ver.

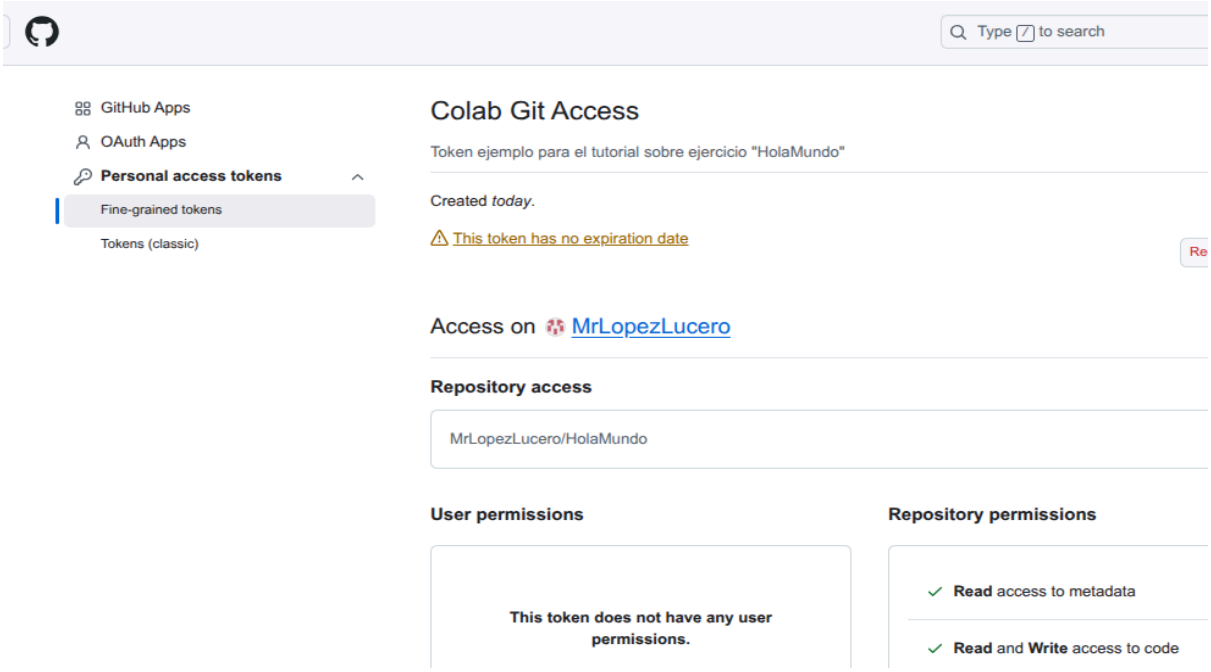


Fig. 3: Captura del Personal Access Token (PAT) generado con éxito
Fuente: Elaboración propia

💡 Consejo:

Guárdalo en un lugar seguro (como un archivo `.txt` temporal o el bloc de notas).
Luego, en Colab, lo usarás así:

```
!git push https://{TOKEN}@github.com/{USER}/{REPO}.git HEAD:main
```

Ejemplo real (reemplazando datos):

```
!git push
https://github_pat_11A1B1CDEFGHijklmnop1234567890ABC@github.com/tuUs
uario/HolaMundo.git HEAD:main
```

4.3) Abrir Google Colab y preparar el entorno

1. Abrí https://colab.research.google.com y creá un nuevo notebook.
2. En la primera celda, chequeá git (opcional):

   !git --version

3. Configurá tu identidad (reemplaza con tu nombre y email):

   !git config --global user.email "tu.email@ejemplo.com"
   !git config --global user.name "Tu Nombre"

Esto hace que los commits tengan autor identificable.

```
!git --version
```

```
git version 2.34.1
```

```
!git config --global user.email "mario.lopez@tupad.utn.edu.ar"
!git config --global user.name "Mario Raúl López"
```

Fig. 4: Captura de las primeras órdenes en Google Colab (paso 4.3)
Fuente: Elaboración propia

4.4) Clonar el repositorio vacío en Colab

En una celda de Colab ejecuta (reemplaza `tu-usuario` y `HolaMundo` por los reales):

!git clone https://github.com/tu-usuario/HolaMundo.git
%cd HolaMundo

Ahora estás dentro de la carpeta del repo en Colab.
Si tu repo fue creado con un README, el `git clone` bajará ese README; si estaba vacío, igual funciona.

```
!git clone https://github.com/MrLopezLucero/HolaMundo.git
%cd HolaMundo
```

```
Cloning into 'HolaMundo'...
warning: You appear to have cloned an empty repository.
/content/HolaMundo
```

Fig. 5: Captura de la directiva de clonación del repo (paso 4.4)
Fuente: Elaboración propia

4.5) Crear el archivo `hola.py`

En Colab podés crear el archivo con `%%writefile`:

%%writefile hola.py
print("¡Hola Mundo desde GitHub y Colab!")

Esto crea `hola.py` con el mensaje.

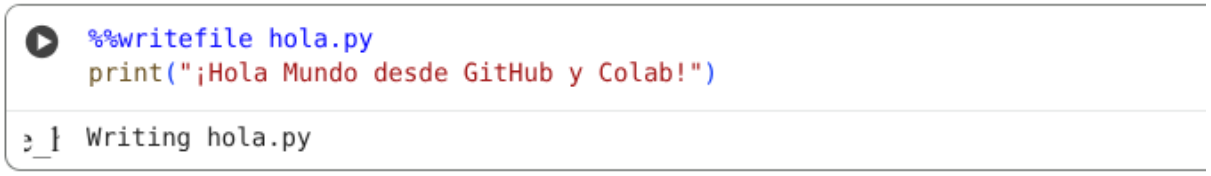


Fig. 6: Captura de la directiva de creación del archivo “hola.py” (paso 4.5)
Fuente: Elaboración propia

4.6) Preparar y confirmar cambios (commit)

!git add hola.py
!git commit -m "Agregar hola.py: primer mensaje Hola Mundo"

Explicación:

- `git add` marca el archivo para el commit (staging).
- `git commit -m` guarda el cambio en el historial local.

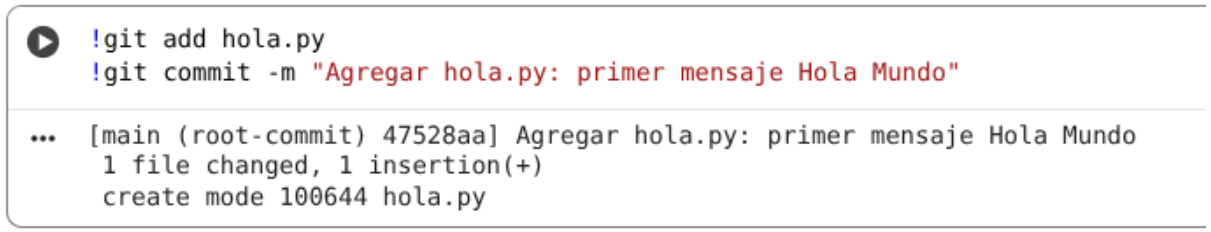


Fig. 7 : Captura del primer commit (paso 4.6)
Fuente: Elaboración propia

4.7) Subir (push) los cambios a GitHub

Usamos el token para autenticarnos al hacer `push`. Reemplaza `{TOKEN}`, `{USER}` y `{REPO}` por tus valores **sin llaves**:

!git push https://{TOKEN}@github.com/{USER}/{REPO}.git HEAD:main

Ejemplo real (supone token `ghp_ABC...` y repo `HolaMundo` y usuario `juan`):
!git push https://ghp_ABCDEF1234567890@example@github.com/juan/HolaMundo.git HEAD:main

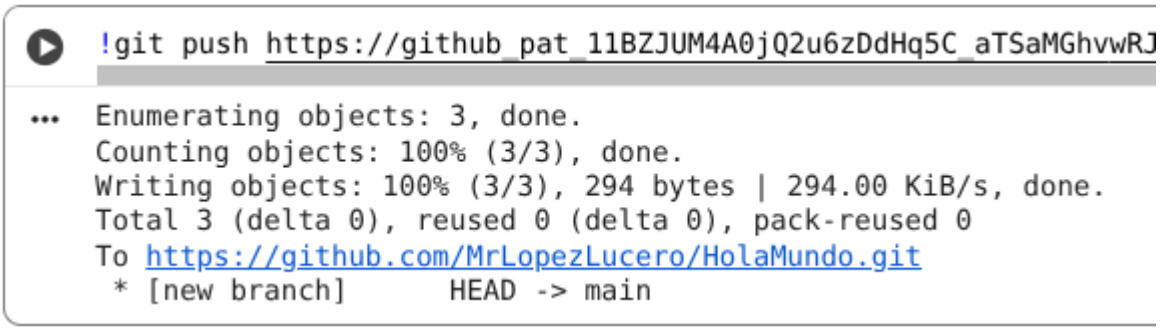


Fig. 8 : Captura del comando push para subir los cambios (paso 4.7)
Fuente: Elaboración propia

Notas:

- Si tu rama principal se llama `master` (raro hoy en día), reemplaza `main` por `master`
- El token en la URL puede quedar en el historial del notebook: **revoca** el token después si te preocupa y crea uno nuevo más seguro.

4.8) Verificar en GitHub

Abrí el repo `https://github.com/tu-usuario/HolaMundo` en el navegador. Deberías ver `hola.py` y el commit en la pestaña **Commits**.

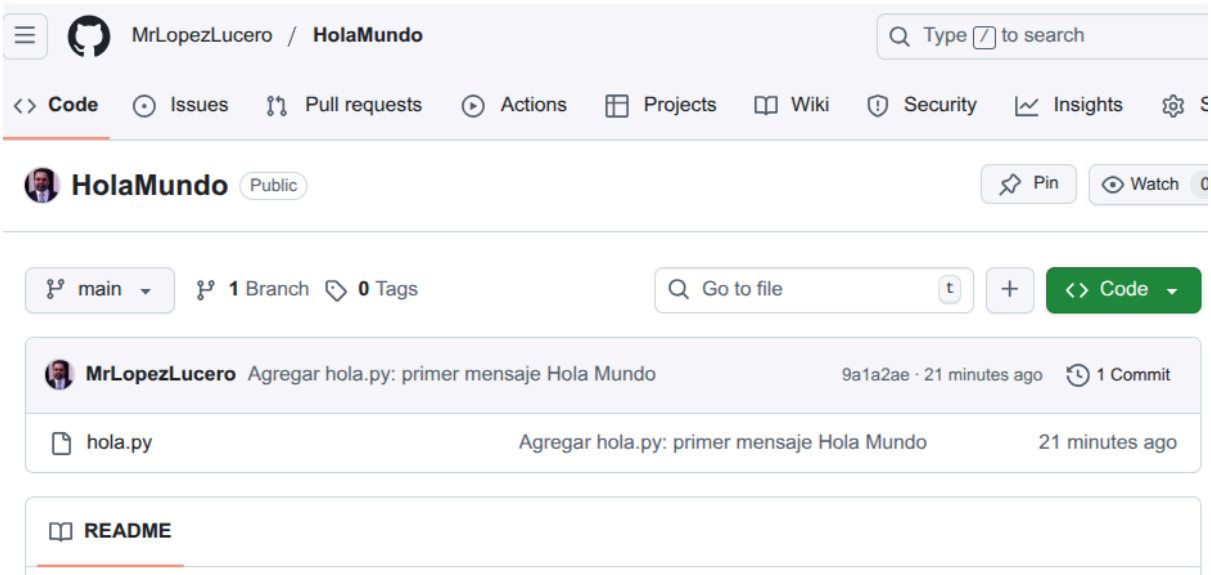


Fig. 9 : Captura de la pantalla de GitHub, verificación (paso 4.8)
Fuente: Elaboración propia

4.9) Ejecutar el programa `hola.py` en Google Colab

Una vez creado y subido el archivo `hola.py` al repositorio, es necesario verificar que el programa funcione correctamente. Google Colab permite ejecutar scripts de Python directamente desde una celda del notebook.
Para ejecutar el archivo, escribí en una celda de Colab el siguiente comando:

!python hola.py

Si el archivo fue creado correctamente, la salida esperada será:

¡Hola Mundo desde GitHub y Colab!



Figura 10 – Ejecución paso a paso del archivo `hola.py` en Google Colab.
Fuente: Elaboración propia

🧩 Recomendación educativa

Después de verificar que tu archivo se ejecuta correctamente, hacé:

```
!git add hola.py
!git commit -m "Verificado hola.py ejecutando correctamente"
!git push
```

Así tu historial de commits reflejará que el código fue probado y confirmado.

4.10) Solución de problemas comunes (Troubleshooting)

Al trabajar con Git y GitHub desde Google Colab, pueden presentarse algunos errores frecuentes. A continuación se detallan los más habituales y sus posibles soluciones.

• Error de autenticación 403/401

Este error indica que el token personal (PAT) es incorrecto o no posee permisos suficientes.

Solución:
Revise que:

- El token esté vigente.
- Tenga permisos repo.

- Lo esté usando en la URL completa del push.
- Error: *failed to push* por rama inexistente

Ocurre cuando el repositorio remoto usa `main` y el local usa `master`, o a la inversa.

Solución recomendada: crear y forzar la rama `main`:

!git branch -M main
!git push https://{TOKEN}@github.com/{USER}/{REPO}.git -u origin main

• Conflictos de fusión (merge conflicts)

En este ejemplo no suelen darse porque solo trabaja una persona.
Sin embargo, en equipos se deben resolver manualmente editando los archivos marcados por Git y haciendo un nuevo commit.

• Buenas prácticas recomendadas

- No comparta su token personal en celdas públicas.
- Use mensajes de commit claros, por ejemplo:
  - ✔️ “Agregar análisis inicial”
  - ❌ “cambios”

En trabajos colaborativos utilice ramas:

git checkout -b nombre-rama

- y gestione Pull Requests en GitHub.
- Ejecute `git pull` antes de comenzar una nueva tarea para mantener el repositorio actualizado.

4.11) Resumen rápido de comandos (Cheat-Sheet)

La siguiente tabla resume los comandos esenciales utilizados en el desarrollo del ejemplo *Hola Mundo* en Google Colab.

Comandos básicos de configuración
!git --version
!git config --global user.name "Tu Nombre"
!git config --global user.email "tu.email@ejemplo.com"

Clonar el repositorio

!git clone https://github.com/tu-usuario/HolaMundo.git
%cd HolaMundo

Crear el archivo `hola.py`
%%writefile hola.py
print("¡Hola Mundo desde GitHub y Colab!")

Ciclo de trabajo con Git
!git add hola.py
!git commit -m "Agregar hola.py: primer mensaje Hola Mundo"
!git push https://{TOKEN}@github.com/{USER}/{REPO}.git HEAD:main

5. Caso real: Hugo, Paco y Luis – Trabajo colaborativo y trazabilidad en equipo

5.1) Contexto del proyecto

El Trabajo Práctico Integrador (TPI) consiste en analizar un conjunto de datos, generar estadísticas descriptivas, producir gráficos en R y elaborar un documento final. Para garantizar un proceso ordenado y reproducible, el equipo utiliza Git para el control de versiones, GitHub como repositorio central y Google Colab como entorno de programación en la nube.

5.2) Organización inicial del repositorio

Hugo crea la estructura inicial del repositorio en GitHub, organizando carpetas claves como `/datos`, `/scripts` y `/resultados`. Además, redacta el archivo `README.md` con las instrucciones básicas de uso y realiza el commit inicial del proyecto. Esta estructura base permite que todos los integrantes trabajen sobre un entorno común y ordenado.

5.3) Desarrollo del análisis en R

Paco desarrolla el análisis estadístico principal utilizando el lenguaje R.
Sus acciones incluyen:

- Clonar el repositorio desde GitHub hacia Google Colab.
- Crear el archivo `analisis.R` dentro de la carpeta `/scripts`.
- Escribir el código necesario para leer datos, procesarlos y generar gráficos.
- Ejecutar el código en Colab usando:

A) Celdas con código R

%load_ext rpy2.ipython
%%R

```
datos <- c(1, 2, 3, 4, 5)
mean(datos)
```

B) Scripts completos

```
!Rscript scripts/analisis.R
```

Luego guarda gráficos y tablas dentro de `/resultados` y registra su progreso mediante commits descriptivos.

5.4) Revisión, documentación y mejora continua

Luis crea una rama específica para documentar y mejorar el proyecto. Sus actividades incluyen:

- Revisar el código desarrollado por Paco.
- Agregar comentarios aclaratorios dentro de `analisis.R`.
- Mejorar el contenido del archivo `README.md`.
- Ejecutar nuevamente los scripts para comprobar que todo funcione correctamente.
- Crear y subir un Pull Request para que el equipo revise y apruebe las mejoras antes de fusionarlas.

Este proceso introduce buenas prácticas de revisión entre pares y control de calidad.

5.5) Integración final del trabajo

El equipo revisa el Pull Request propuesto por Luis y, una vez aprobado, lo fusiona con la rama principal (`main`). Los gráficos generados se almacenan en la carpeta `/resultados` y se prepara el documento final del TPI.
Gracias al uso de Git y GitHub, todo el trabajo queda registrado de manera ordenada y reproducible.

5.6) Beneficios del enfoque colaborativo

El caso de Hugo, Paco y Luis demuestra cómo el uso organizado de Git y GitHub contribuye a:

- Mantener un historial de versiones completo.
- Evitar la pérdida de información.
- Facilitar el trabajo simultáneo sin sobrescribir archivos.
- Integrar mejoras mediante Pull Requests.
- Asegurar la trazabilidad total del proyecto.
- Permitir reproducir el análisis en cualquier equipo mediante Google Colab.

Este enfoque replica las metodologías utilizadas en entornos profesionales de desarrollo de software y análisis de datos.

6. Buenas prácticas – Documentación, seguridad y comunicación en proyectos

El uso de Git y GitHub en proyectos académicos y profesionales no solo implica conocer comandos o flujos de trabajo, sino también adoptar **buenas prácticas** que garanticen claridad, seguridad, eficiencia y una comunicación fluida entre los integrantes del equipo. Este apartado reúne las recomendaciones esenciales que permiten sostener proyectos ordenados, legibles y escalables, tanto en el ámbito educativo como laboral.

6.1) Documentación clara y actualizada

La documentación es un componente central para asegurar que cualquier integrante del equipo —actual o futuro— pueda comprender la estructura, el propósito y la evolución del proyecto. Para ello, se recomienda:

- Mantener un **README.md completo y actualizado**, que incluya:
  - Objetivo del proyecto.
  - Requisitos del entorno.
  - Instrucciones para ejecutar el código.
  - Estructura de carpetas.
- Utilizar comentarios en el código que expliquen:
  - Variables importantes.
  - Funciones y bloques críticos.
  - Supuestos, decisiones y justificaciones.
- Registrar cambios significativos en un archivo `CHANGELOG.md` o mediante commits descriptivos.

Una buena documentación reduce la curva de aprendizaje y facilita la continuidad del proyecto.

6.2) Seguridad y protección de credenciales

En entornos colaborativos es fundamental proteger información sensible. Las prácticas recomendadas incluyen:

- **Nunca compartir tokens personales (PAT)** en capturas de pantalla, archivos o celdas públicas.
- **No subir contraseñas, claves o archivos privados** al repositorio.
- Utilizar `.gitignore` para evitar que se suban archivos que contienen datos sensibles.
- Revocar inmediatamente un token que haya sido expuesto accidentalmente.
- Utilizar tokens únicamente con permisos mínimos necesarios (principio de “privilegios mínimos”).
- Evitar hacer `push` desde redes públicas sin cifrado adecuado.

La seguridad no solo protege información personal, sino también la integridad del proyecto.

6.3) Consistencia en los commits y el historial de versiones

Un historial de commits prolijo permite comprender el avance del proyecto y facilita la detección de errores o retrocesos. Se recomienda:

- Utilizar **mensajes de commit claros, breves y significativos**, por ejemplo:
  - ✔️ “Agregar función para cálculo de media en R”
  - ✔️ “Actualizar README con instrucciones de ejecución”
  - ❌ “Arreglos varios”
  - ❌ “prueba”
- Realizar commits **pequeños y frecuentes**, en lugar de grandes cambios sin explicación.
- Utilizar ramas para contener funcionalidades nuevas o experimentales.
- Mantener la rama `main` siempre funcional y libre de errores críticos.

6.4) Comunicación efectiva dentro del equipo

En proyectos colaborativos, el código no es suficiente: la comunicación constante es clave. Las recomendaciones incluyen:

- Utilizar los **Issues de GitHub** para reportar errores, proponer mejoras o documentar decisiones importantes.
- Escribir descripciones claras al crear un **Pull Request**, explicando:
  - Qué se cambió.
  - Por qué.
  - Cómo evaluarlo o reproducirlo.
- Respetar los tiempos de revisión de los compañeros.
- Utilizar etiquetas (“labels”) para organizar tareas:
  - *bug*, *enhancement*, *documentation*, *help wanted*, etc.
- Mantener un acuerdo de estilo y convenciones del equipo (nombres de archivos, formatos, estructura del código).

6.5) Organización del proyecto y manejo de ramas

La creación de ramas permite experimentar y trabajar en paralelo sin afectar el proyecto principal. Buenas prácticas incluyen:

- Crear ramas específicas por tarea o funcionalidad:

```
 git checkout -b analisis-graficos
git checkout -b documentacion
```

- Mantener nombres consistentes:

    - `fix/…` para correcciones
    - `feature/…` para nuevas funciones
    - `docs/…` para documentación
- Fusionar cambios a `main` únicamente después de revisiones y validaciones (Pull Requests).
- Eliminar ramas obsoletas para mantener el repositorio ordenado.

6.6) Reproducibilidad del análisis

Un proyecto bien organizado debe poder reproducirse desde cero en cualquier equipo. Para ello:

- Incluir instrucciones exactas para ejecutar el código.
- Señalar la versión de R, Python o bibliotecas utilizadas.
- Mantener archivos de datos limpios y debidamente identificados.
- Evitar dependencias locales o rutas absolutas.

La reproducibilidad es especialmente importante en proyectos académicos y científicos.

6.7 Conclusión del apartado

La aplicación sistemática de buenas prácticas mejora la calidad del proyecto, reduce errores, facilita el trabajo en equipo y permite sostener el desarrollo a lo largo del tiempo. Adoptar estas pautas desde etapas tempranas de la formación profesional contribuye a preparar a los estudiantes para entornos laborales reales, donde la documentación, la seguridad y la comunicación efectiva son tan importantes como el código mismo.

7. Conclusiones y bibliografía

7.1) Conclusiones

El uso integrado de Git, GitHub y Google Colab constituye una herramienta fundamental para el trabajo colaborativo en la Tecnicatura Universitaria en Programación a Distancia de la UTN. Su aplicación favorece:

- La organización del proyecto, mediante control de versiones y trazabilidad completa.
- La colaboración efectiva, gracias al uso de ramas, commits descriptivos e integración mediante Pull Requests.
- La reproducibilidad del análisis, independizando el trabajo del entorno de ejecución individual.
- La documentación clara, que permite transmitir conocimientos y sostener proyectos en el tiempo.

- La profesionalización del trabajo académico, replicando metodologías utilizadas en organizaciones reales de desarrollo de software y análisis de datos.

El caso práctico de Hugo, Paco y Luis demuestra que estos instrumentos no solo ordenan el trabajo, sino que también mejoran la comunicación, reducen errores y permiten que cada integrante agregue valor desde su rol.

En síntesis, dominar Git y GitHub prepara al estudiante para integrarse con solvencia a equipos de desarrollo modernos, promoviendo competencias técnicas y colaborativas imprescindibles en el mundo laboral actual.

7.2. Bibliografía

Bibliografía general

- Chacon, S., & Straub, B. (2014). *Pro Git* (2nd ed.). Apress. Disponible en: https://git-scm.com/book/en/v2
- GitHub Docs. *Managing branches, repositories and pull requests*. Disponible en: https://docs.github.com/
- Google Colab. *Using Colab with GitHub*. Disponible en: https://colab.research.google.com/
- Wickham, H. (2017). *R for Data Science*. O'Reilly Media. Disponible en: https://r4ds.hadley.nz/

Normativa y guías recomendadas

- Free Software Foundation. (2020). *GNU General Public License*. https://www.gnu.org/licenses/gpl-3.0.html
- The Linux Foundation. (2023). *Open Source Best Practices*. https://www.linuxfoundation.org

Sitios oficiales de referencia

- Sitio oficial de Git: https://git-scm.com
- Sitio oficial de GitHub: https://github.com
- Sitio oficial de Python: https://www.python.org
- Sitio oficial del lenguaje R: https://www.r-project.org

7.3. Créditos y agradecimientos

Esta guía fue elaborada por:

Prof. Mario Raúl López
Cátedra Organización Empresarial

Tecnicatura Universitaria en Programación a Distancia
Universidad Tecnológica Nacional (UTN)

Con asistencia técnica y generativa de:

- **ChatGPT (OpenAI)**
- **Gemini (Google DeepMind)**

cuya colaboración permitió integrar procesos, generar ejemplos, validar contenidos y preparar materiales complementarios para estudiantes.

Agradecimiento especial al **equipo docente de la cátedra Organización Empresarial**, liderado por la **Prof. Gabriela Martínez**, cuyo compromiso académico y orientación pedagógica hicieron posible la elaboración y revisión integral de esta guía.